MARKETMIND AI

Milestone 3 and 4 Workflow Plan

A practical continuation of the completed authentication, business operations, customer segmentation and forecasting platform.

CURRENT BASELINE	MILESTONE 3	MILESTONE 4
Milestones 1 and 2 complete FastAPI + React + RBAC + database Segmentation and forecasting	Recommendations Churn prediction ML anomaly detection	Testing and hardening Deployment and monitoring UAT and final documentation

Planning rule: no dashboard may show a model-generated insight until the tenant has enough eligible data and the model passes its defined quality gate.

1. Milestone 3 workflow

Goal: add customer retention, product recommendation and anomaly intelligence without breaking the existing tenant, store, seller and Administrator security model.

Confirm definitions > **Prepare data** > **Build features** > **Train and compare** > **Publish safely**

Step	Work	Expected result
1	Freeze the business definitions	Agree churn inactivity window, eligible customers, recommendation context, anomaly severity and alert owners.
2	Extend data contracts	Add consent, dated engagement events, customer-product baskets, inventory movements and feedback fields without changing existing IDs.
3	Build reusable features	Reuse RFM, engagement, segment, forecast residual and stock signals; add point-in-time churn labels and customer-item interaction features.
4	Establish simple baselines	Use inactivity rules, popular/category products, association rules and statistical thresholds before evaluating complex models.
5	Train and evaluate	Use chronological splits, compare candidates, inspect role/store slices and reject leakage, unstable results or excessive false alerts.
6	Store versioned outputs	Persist model run, feature version, scope, metrics, prediction time and customer/product/alert outputs. Never overwrite actual records.
7	Connect APIs and dashboards	Expose role-scoped results, clear not-ready states, explanations, feedback and refresh controls through FastAPI and React.
8	Run acceptance checks	Repeat imports, tenant-isolation tests, model gates, API contracts, frontend states and human review of recommendations and alerts.

Recommended build order
Churn labels and datasets first; then association-rule recommendations; then churn candidates; then Isolation Forest anomaly scoring. This order gives the backend and frontend stable response contracts before adding heavier models.

2. Dataset plan

The current datasets remain useful. Milestone 3 should extend them with governed production fields instead of replacing the Milestone 1 and 2 pipelines.

Dataset	Status	Important fields	How MarketMind uses it
Online Retail II transactions	Already used	Invoice, customer, product, quantity, price, date and returns	Churn features; customer-item baskets; association rules; repeat-purchase analysis
Customer summary and segment assignments	Already used	RFM, tenure, purchase gaps, engagement, return rate and segment	Strong starting features for churn; segment-aware recommendations and retention actions
Application sales and sales lines	Already used	Tenant, store, seller, customer, SKU, quantity, amount, status and date	Tenant-specific churn history, basket building, cross-sell evidence and sales anomalies
Product and inventory records	Already used	SKU, category, availability, stock, reorder level and store	Remove unavailable products; stock-aware recommendations; inventory anomalies
Forecast outputs and actuals	Already used	Actual, predicted, bounds, residual, model version and horizon	Detect unusual revenue/demand deviations and support model monitoring
Customer engagement events	Add for M3	Visit/contact/campaign timestamps, channel, outcome and consent	Reliable churn labels, engagement trends and recommendation feedback
Recommendation feedback	Add for M3	Shown, clicked, accepted, rejected, purchased and timestamp	Offline/online evaluation and later personalization
Operational telemetry	Add for M4	API latency/errors, audit events, model runs, drift and job status	Deployment health, incident response and retraining decisions

Data rules that prevent misleading AI

- Use stable tenant, store, customer, transaction and product identifiers; never guess cross-dataset mappings.
- Build churn labels from a documented observation window and a later inactivity window; do not use future information in training features.
- Keep returns, voids, unavailable products and stockouts explicit so models do not learn false demand or revenue.
- If consent, history or interaction density is insufficient, return a not-ready state and an exact data requirement.
- Keep full generated artifacts outside Git; version code, schemas, deterministic samples and aggregate reports.

3. Model plan and compatibility

Every new model is designed to reuse the current preprocessing, model-run metadata, database imports, FastAPI scopes and React report patterns.

Capability	Decision	Model	Why compatible	Acceptance evidence
Customer segmentation	Continue	K-Means primary; Hierarchical comparator	Supplies RFM/engagement groups and segment history to churn and recommendations	Silhouette, Davies-Bouldin, stability, useful profiles
Revenue and demand forecasting	Continue	Seasonal Naive, Linear Trend/Prophet, XGBoost, Random Forest	Forecast residuals become anomaly signals; demand remains linked to inventory	MAE, RMSE, bias, baseline improvement, matured accuracy
Churn prediction	New in M3	Logistic Regression baseline; Random Forest and XGBoost candidates	Tabular customer features already exist; chronological labels and calibration must be added	Recall, precision, F1, PR-AUC, ROC-AUC, calibration and slice checks
Product recommendation	New in M3	Association rules first; popularity/category fallback; item-based CF or implicit ALS when data is dense	Invoice-product baskets and SKU records already exist; availability filtering uses inventory	Precision@K, Recall@K, coverage, diversity and reviewed conversion
Anomaly detection	New in M3	Business thresholds and robust statistics first; Isolation Forest candidate	Current alerts, forecast residuals, sales lines and inventory data provide evidence	Reviewed precision, detection rate, false-positive workload and time to resolve

Model selection rule
A complex model is selected only when it beats the approved baseline on unseen, time-ordered data and remains understandable at the permitted user scope. Otherwise MarketMind keeps the baseline, displays the limitation and records the failed candidate for review.

Cold-start behavior

- New business: show onboarding readiness and data requirements; do not copy demo predictions.
- New customer: use safe popularity/category suggestions until purchase history is sufficient.
- Sparse churn history: show rule-based inactivity status, not an invented probability.
- Unmapped product or store: keep mapping_status as unmapped and stock risk as unknown.

4. Application integration

Milestone 3 extends the existing architecture; the frontend never reads a model artifact or the database directly.

React role view > **FastAPI + RBAC** > **Model service** > **SQLAlchemy** > **Versioned records**

Module	Proposed API group	Response responsibility
Churn	GET /api/v1/churn/summary GET /api/v1/churn/customers POST /api/v1/churn/train	Summary, scoped risk list, drivers, model metadata and not-ready reason
Recommendations	GET /api/v1/recommendations/customer/{id} GET /api/v1/recommendations/product/{id} POST /api/v1/recommendations/feedback	Ranked eligible products, reason, score, model/rule version and feedback receipt
Anomalies	GET /api/v1/anomalies PATCH /api/v1/anomalies/{id} POST /api/v1/anomalies/train	Evidence, severity, status, scope, assignee, detector version and resolution notes
Monitoring	GET /api/v1/models/monitoring GET /api/v1/models/{id}/runs	Readiness, quality, drift, errors, last run and approved/active state

Role access carried forward

Role	Churn	Recommendations	Anomalies	Restriction
Business Owner	Tenant churn summary and permitted customers	Business/customer recommendations	Tenant sales and stock alerts	No model configuration
Store Manager	Assigned-store view	Store/customer/product recommendations	Assigned-store alerts and workflow	No user or model administration
Sales Executive	No churn report	Assigned-customer recommendations only	Only alerts tied to own work if permitted	No inventory/model controls
Administrator	All permitted scopes with MFA	Configure, monitor and audit	Configure detectors and audit response	Internal platform account only

Database additions

- churn_model_runs and churn_predictions with label policy, probability, band, drivers and scope
- recommendation_model_runs, recommendation_results and recommendation_feedback with eligibility evidence
- anomaly_model_runs, anomaly_events and anomaly_actions with severity, state, assignee and audit trail
- shared model status, version, feature schema, metrics, approval, active flag and generated-at fields

5. Milestone 4 workflow

Goal: turn the integrated local application into a tested, observable and recoverable release. Milestone 4 continues all approved models; it does not introduce a separate business model.

Step	Workstream	Completion evidence
1	Freeze release scope	Tag approved requirements, API schemas, database migrations, model versions and known limitations.
2	Run clean-install tests	Create an empty PostgreSQL database, apply every Alembic migration, seed roles, import data twice and verify no duplicates.
3	Functional and role testing	Test onboarding, sales, inventory, profiles, settings, segmentation, forecasts, churn, recommendations and anomaly workflows for all four roles.
4	Model validation	Reproduce features and artifacts, check leakage, baseline comparison, slices, calibration, false alerts, rollback and actual-vs-predicted monitoring.
5	Security and privacy	Test authentication, MFA, session revocation, tenant isolation, uploads, rate limits, secrets, audit coverage and PII minimization.
6	UX, accessibility and performance	Check responsive screens, keyboard access, contrast, empty/error/loading states, API latency, imports and concurrent jobs.
7	Container and staging release	Build Docker images, run FastAPI with PostgreSQL, apply migrations automatically, configure HTTPS/secrets and perform smoke tests.
8	Operations and recovery	Enable logs, health checks, alerts, backups, restore rehearsal, model rollback, application rollback and incident runbooks.
9	UAT and final handover	Run role-based demo scenarios, record acceptance, publish documentation and release only after the rollback plan is proven.

Recommended deployment path
Dockerized FastAPI + PostgreSQL in staging first, then an approved cloud target such as Render or Railway. Keep frontend and backend environment values separate, run migrations before smoke tests, and retain the previous application and model versions for rollback.

6. Priority, ownership and completion gates

Suggested task order for the team

Priority	Area	Main work	Why now
P0	Data and product decisions	Churn policy, consent, event schema, baskets, anomaly severity and success metrics	Blocks trustworthy training
P1	Data pipeline	Validation, point-in-time features, interaction density, lineage and quality reports	Blocks all three M3 models
P2	Models	Baselines, candidates, chronological evaluation, model cards and safe fallbacks	Produces approved artifacts
P3	Backend/database	Migrations, imports, APIs, RBAC, audit, jobs and model lifecycle	Makes outputs usable and secure
P4	Frontend	Role pages, filters, explanations, feedback, alerts and not-ready states	Makes results understandable
P5	Integration and QA	Clean install, repeat imports, role tests, model tests and end-to-end demo	Completes Milestone 3
P6	Deployment hardening	Security, accessibility, load, monitoring, backups, UAT and rollback	Completes Milestone 4

Definition of done

Milestone	Release gate
Milestone 3	Three modules use real tenant data; baselines and candidates are documented; versioned results reach role-scoped APIs and dashboards; low-data states show no fake predictions; automated model/API/RBAC tests pass.
Milestone 4	Clean PostgreSQL install passes; all four role journeys pass; no unresolved critical/high security issue; load and accessibility targets are accepted; monitoring, backup/restore and rollback are demonstrated; UAT and release documents are signed.

What continues from the current project
FastAPI routing, Pydantic validation, SQLAlchemy/Alembic, JWT + MFA, backend RBAC and data scope, onboarding imports, React role dashboards, customer features, K-Means segmentation, forecasting, model metadata, tests and Docker/PostgreSQL configuration. New work should extend these patterns, not create a separate AI application.

Prepared from the MarketMind project brief, SRS v1.1 and the current Garvitk001/Review implementation.